NimbusPay ticket triage: fine-tuning Qwen2.5-1.5B-Instruct (report)

Every number was copied from notebook outputs (fresh-T4 final run and seed check, 2026-11-12/13) or from the experiment log; non-T4 numbers are labelled.

1. Summary

`Qwen/Qwen2.5-1.5B-Instruct`, LoRA r=16 on all linear layers (18.46M trainable parameters, cap 50M), own PyTorch + PEFT loop, fp16 on a free Colab T4. An audit turned 1,445 training rows into 1,320 clean rows (worth about +10 points). The final model (clean data, 5 epochs) scores **94.0% exact match / 98.9% mean field accuracy** on the 200 dev tickets (strict `score.py`; 93.5% exact on the 170 clean dev tickets) in a fresh top-to-bottom run of the notebook. Two more seeds of the same configuration gave 94.0 and 92.0 (mean 93.3, sd 1.15). All 400 test predictions are valid 8-key JSON. Base model, fixed prompt: 0%; with all rules in the prompt: 0% exact.

2. Data audit (Part A)

Method: structural profile; sentence-template mining; a rule checker that re-derives every deterministic field from the ticket text using only SCHEMA.md; a 5-fold cross-validated classifier for category; duplicate and dev-leakage checks. Code `audit/`; log `cleaning_log.csv` (300 rows: 125 dropped, 175 fixed, 1,320 kept).

Problem	Rows	Action
Label JSON truncated	25	drop
Legacy v1 schema (`prio` low/medium/high, `dd/mm/yyyy`)	70	fix (mapping equals the spec-derived priority on 70/70)
Junk tickets (`test`, `asdf asdf`, empty) with filled labels	10	drop
Exact duplicates	60	drop second copy
Train tickets that are exact copies of dev tickets (leakage; none overlap test)	30	drop
`amount_paise` stored in rupees (incl. `k`, `lakh`)	25	fix
`txn_id` copied raw (`np 81633 47892`)	20	normalise
`priority` contradicts the spec	60	fix; `needs_human` equals the derived value in 60/60

Checked and clean: category (CV agreement 97.9%; all 29 disagreements read by hand, none wrong), channel, language, dates, `needs_human`, formatting. The checker reproduces the dev gold labels on all 7 deterministic fields at 100% (dev labels used for measurement only; dev inputs, never labels, found the 30 leaked rows).

Not caught (found in a review after I submitted; not retrained): 7 train rows are copies of dev tickets that differ only in whitespace (tr-00343, 358, 583, 588, 616, 970, 1031 = dv-00039, 51, 198, 52, 80, 85, 77), and 17 more whitespace-variant duplicate rows stay in the clean set. `cleaning_log.csv` describes exactly what was trained on. The 7 dev copies are all scored correctly; without them clean-dev exact is 152/163 = 93.3% instead of 93.5%.

3. Pipeline (Part C)

- Stack: plain PyTorch + Hugging Face PEFT, own loop (no Trainer/TRL): stable while Colab upgrades libraries; loss only at assistant positions, so 152k-vocab logits are never built for prompt tokens (a 2,200-token ticket would need about 1.3 GB of fp32 logits); every line explainable live. Qwen's template has a system role, used as given.
- The model's own chat template on `[system, user]` + generation prompt; target = JSON + `<|im_end|>`; loss only on the completion (my loss equals the Hugging Face loss within 1e-4 in the CPU smoke test `tests/smoke_test_train.py`).
- Sequence length from evidence: 97% of examples are at most 263 tokens, but 38 quoted-thread tickets are 1,000-2,214 tokens and the label is last, so truncation would delete the target: `MAX_LEN = 2560`, nothing truncated; fp16 autocast with fp32 LoRA weights, gradient checkpointing.
- Budget: r=8/16/32/64 = 9.2/18.5/36.9/73.9M parameters, so r=64 breaks the 50M cap. AdamW, lr 2e-4, cosine, 5% warmup, 16 sequences per step; greedy decoding with the model's own generation_config (`repetition_penalty=1.1`).

4. Baselines (Part B, dev, strict)

	exact	field acc	valid JSON
B1 base model, fixed prompt	0.0%	0.0%	0% (own keys, JSON in code fences)
B2 base model, SCHEMA.md rules in prompt	0.0%	19.4%	34% (lenient parse: field 59.9%, exact 0.5%)

5. Controlled experiments (Part D)

Colab T4, same 200 dev tickets (one ticket = 0.5 points). "rep 1.0" = scored with `repetition_penalty=1.0` instead of the default 1.1.

Run (one change vs R0)	exact	field	clean-dev	rep 1.0	peak GPU	train min
R0 clean data, 16-bit LoRA, 3 epochs, seed 0	89.0	97.9	88.8	90.5	5.3 GB*	14.3
D1 raw uncleaned data	79.0	96.5	77.6	87.0	5.3 GB	15.4
D2 4-bit QLoRA	88.0	97.7	88.2	90.5	4.7 GB	17.7
D3a 1 epoch	42.0	90.1	42.4	48.0	5.3 GB	4.8
D3b 5 epochs	93.0	98.6	92.4	93.0	5.4 GB	23.8
D4 priority, needs_human emitted last	87.0	97.5	87.6	89.0	5.3 GB	14.3
Fresh final notebook: R0 re-run	89.0	97.8	88.8	90.0	5.33 GB	14.5
Fresh final notebook: **FINAL, seed 0 (submitted)**	**94.0**	**98.9**	**93.5**	94.0	5.4 GB	24.1
Seed check: FINAL settings, seed 1	94.0	98.9	94.1	94.0	5.35 GB	22.1
Seed check: FINAL settings, seed 2	92.0	98.5	91.8	92.0	5.36 GB	22.2

*R0's first log said 8.2 GB, inflated by about 3 GB left over from a failed attempt; the live re-run measured 5.33 GB. Same seed on a fresh runtime is not bit-identical (R0 field 97.8 vs 97.9; D3b settings 94.0 vs 93.0, 2 tickets). Non-T4 repeat (college PC, RTX PRO 4000 24 GB, torch 2.11; **memory and time not comparable**; files in `college/results/`): R0 seed 0 89.0, R0 seed 1 91.5, D3b 92.0, D4 87.5. The seed check (`seeds_notebook.ipynb`, same library versions, dev only) ran after the submitted adapter was fixed; the submission is seed 0 and was not picked as the best of three.

Findings. 1. **Cleaning: +10 points** (D1), far above noise; the raw-trained model is also fragile to the decoding config (+8 points without the repetition penalty vs +1.5 for the clean model). 2. **QLoRA** costs about 1 point (within noise), saves 0.6 GB and is 24% slower; the saving is small, probably because activations and the fp32 upcast dominate (not measured separately). 3. **Epochs.** My hypothesis (one epoch is enough) was wrong: after 1 epoch the loss (0.024) looks small but exact match needs every digit right; it reaches ~0 only after 3-5 epochs (final loss 0.0240 / 0.0013 / 0.0002). On dev, 5 epochs also removes the repetition-penalty sensitivity (94.0 with and without it), but on test 12 of 400 outputs still differ at `repetition_penalty=1.0`. The final model uses 5 epochs by a rule I wrote down before D3b finished (clean-dev exact must beat R0 by 2 points: 92.4 vs 88.8; the repo cannot prove the timing). The three seeds of the final settings give exact mean 93.3 (sd 1.15, range 92.0-94.0) and field accuracy mean 98.75 (sd 0.22); all five 5-epoch runs I have (92.0-94.0, T4 and college) are above all four 3-epoch runs (89.0-91.5), though repeats of one seed are not independent. 4. **D4 (my idea) is a negative result** (87.0 T4, 87.5 college): priority errors are not caused by deciding before writing the amount.

6. Error analysis (Part E, final model, seed 0, 200 dev tickets)

12 of 200 tickets have a wrong field (6 with one, 6 with two): priority 8, category 4, needs_human 4, amount_paise 1, txn_date 1, none in txn_id, channel, language. Per-field accuracy (derived from these counts): priority 96.0, category 98.0, needs_human 98.0, amount 99.5, date 99.5, the other three 100. Against the live R0 (22 wrong): only R0 wrong 11, only FINAL wrong 1, both wrong 11, both right 177. The 12 failures (my reading, not proof):

id	wrong	gold -> predicted	why (likely)
dv-00017	amount	18082725 -> 18082525	"Rs 1,80,827.25": one digit mis-copied; exact match needs every digit
dv-00019	category	refund -> `product_fault`	value not in the schema (scorer checks types only, so a wrong value, not invalid JSON); "product was faulty", no refund word
dv-00037	priority, needs_human	P1 -> P2, true -> false	"284669 rupees" without separators: the Rs 50,000 rule missed
dv-00046	category, priority	fraud -> refund, P1 -> P3	"a payment on my statement is wrong", no "I did not make this"; fraud wins, priority follows
dv-00056	priority, needs_human	P2 -> P1, false -> true	Rs 44,206.75 is under Rs 50,000 but looks large

id	wrong	gold -> predicted	why (likely)
dv-00090	txn_date	2026-07-10 -> 2026-07-11	Hinglish "parso" (day before yesterday) read as "kal"
dv-00127	priority	P3 -> P2	Rs 4,836 just under the Rs 5,000 line; distractor "account balance Rs 111"
dv-00129	priority, needs_human	P2 -> P1, false -> true	Rs 49,657 just under the Rs 50,000 line
dv-00148	priority	P2 -> P1	Rs 42,062 refund, "second complaint", long quoted thread (rare: 38 in train)
dv-00157	category, priority	fraud -> account_access, P1 -> P2	"OTP maang ke paise nikal liye": the OTP word pulled it to account_access; money taken without consent is fraud
dv-00177	category	payment_failed -> fraud	distractor "a red flag on my top up"; the top-up failed (P1 either way)
dv-00180	priority, needs_human	P3 -> P1, false -> true	amount copied right (313000 paise) but priority driven by distractor "account balance Rs. 241670"

Patterns: 8 of 12 involve priority, 6 of those 8 near the Rs 5,000 / 50,000 thresholds or a distractor number, the other 2 follow a wrong category; all 4 needs_human errors follow a wrong priority; the 4 category errors are confusions between fraud, refund, payment_failed and account_access, one of them an out-of-schema value; only 2 tickets have an error in a copied field. By gold category, tickets with a wrong field: refund 6/55, payment_failed 4/53, fraud 2/15, kyc 0/24, account_access 0/19, offers 0/18, other 0/16. Surface forms are learned; what is left is threshold logic and Hinglish phrasing.

Test predictions (400 rows, checked against SCHEMA.md without labels; none exist): all valid 8-key JSON, ids in test order. Five rows break a schema rule: ts-00001 `txn_date` `2026-07-00` (non-existent day; "Received 2026-07-06, 6 din pehle" should be 2026-06-30); ts-00209 `txn_id` `NP12077555550` has 11 digits (ticket: `np 12077 55550`); ts-00367 `channel` `apple_pay` (not in the enum); ts-00008 and ts-00377 category `other` with non-null transaction fields (`other` itself is doubtful; no label to prove it). A further 9 rows have a priority that contradicts the model's own category and amount (ts-00016, 00046, 00067, 00181, 00284, 00303, 00367: P2 although the amount is under Rs 5,000 or null; ts-00065, 00269: P3 although the amount is Rs 5,000 or more). So at least 14 of 400 rows (3.5%) are visibly wrong without any labels; dev (94.0%) probably overstates test, which has more unseen templates, and I cannot measure the test score. The submission is raw model output, so none were repaired.

7. What I would do with one more week

(a) Augment from the spec's own variation list (ids, dates, channels, legal threats): test inputs contain many sentence templates that never occur in train (38% vs 24% for dev, measured on inputs only); (b) loss weighting or a curriculum on the Rs 5,000 / 50,000 thresholds; (c) 5+ seeds per variant; (d) a held-out "hard slice" of paraphrases; (e) tickets with relative dates across month boundaries ("6 din pehle" on the 6th), ids written with spaces, and enum-valid channels (the 5 test rows above); (f) a rule-based date and amount verifier at evaluation time only, to measure what it would catch (not part of the submission).

8. AI tools and honesty notes

An AI coding assistant (an LLM agent) wrote the audit scripts, notebooks, training module and tests, and drove the Colab runs through browser automation; I reviewed them and must be able to defend every line and number. A second session of the same assistant on a college PC ran the non-T4 repeats with a prepared script. Notes: (1) I inspected test *inputs* (never labels) to measure the paraphrase shift and to check the predictions against the schema; no test row was used for training or tuning. (2) Dev inputs found the 30 leaked rows; dev labels only validated the checker. (3) The college `results.jsonl` (`college/results/`) keeps one stale error row from a failed first attempt (missing `jinja2`). (4) The submitted predictions use the default generation config, plain `generate`, batch size 1 (33.4 min on the T4); they equal batch-size-16 output on 400/400 tickets and `repetition_penalty=1.0` output on 388/400. (5) The submitted model is one seed of three; dev has 200 tickets, so 94.0 vs 93.0 is 2 tickets.